

Library Tasks in TornadoVM

The hybrid API surface, its execution mechanism, and an exhaustive comparison against the NVIDIA libraries it binds

6	39	75	0
providers shipped cuBLAS · cuBLASLt · cuFFT · cuDNN · cuSPARSE · CUTLASS	public factory entry points across all providers	JUnit tests in <code>tornado-unittests</code>	core-runtime changes needed to add a provider

A **library task** is a node in a TornadoVM `TaskGraph` that dispatches a native NVIDIA library call instead of a JIT-compiled kernel. It shares TornadoVM-managed device buffers with the surrounding JIT tasks, runs on the same CUDA stream, and is captured into CUDA Graphs alongside them — so a value produced by a `@Parallel` kernel feeds cuBLAS (and back) with no host round trip and no manual memory management.

This report is derived entirely from the source tree at commit `e06a2e5` (branch `claude/tornadovm-library-support-doc-mjrg18`): the six `tornado-<lib>` modules, the two JNI modules under `tornado-drivers/`, the runtime SPI in `tornado-runtime`, and the interpreter's library-launch path. Where the repository's own documentation and its code disagree, the code wins, and the discrepancy is flagged.

CONTENTS

Part I — Mechanism

1. The `libraryTask` API surface
2. Anatomy of a library task
3. The provider SPI
4. Device pointers, layout, alignment
5. Stream binding and ordering
6. `prepare()` and CUDA Graph safety
7. Context lifetime and caching
8. Profiling and observability
9. Execution-plan controls
10. Failure modes

Part II — What is supported

11. cuBLAS
12. cuBLASLt
13. cuFFT
14. cuDNN
15. cuSPARSE
16. CUTLASS
17. Composition patterns

Part III — Comparison with NVIDIA

18. Method and the two questions
19. cuBLAS vs. cuBLAS
20. cuBLASLt vs. cuBLASLt
21. cuFFT vs. cuFFT
22. cuDNN vs. cuDNN
23. cuSPARSE vs. cuSPARSE
24. CUTLASS vs. CUTLASS
25. Semantic fidelity gaps
26. Cross-cutting structural limits
27. Libraries with no binding

Part IV — Performance

28. Measured numbers
29. Where parity holds and breaks

Part V — Engineering surface

30. Build, install, packaging
31. Test coverage
32. Writing a provider

Part VI — Assessment

33. Findings

Source: `beehive-lab/TornadoVM` @ `e06a2e5` · Reference GPU for all measurements: NVIDIA GeForce RTX 4090 (Ada, SM 8.9), CUDA 12.6, cuDNN 9.23 · Requires the CUDA backend (`make BACKEND=cuda`).

Part I — Mechanism

1. The `libraryTask` API surface

The entire user-facing addition to TornadoVM is one method on `TaskGraph`, a sibling of `.task(...)`:

```
TaskGraph graph = new TaskGraph("hybrid")
    .transferToDevice(DataTransferMode.EVERY_EXECUTION, matrix, vector)
    .task("pre", MyKernels::preprocess, matrix) // JIT
    .libraryTask("sgemv", CuBlas::cublasSgemv, // native cuBLAS
        CuBlasOperation.CUBLAS_OP_T.operation(),
        m, n, 1.0f, matrix, lda, vector, 1, 0.0f, output, 1)
    .task("post", MyKernels::postprocess, output) // JIT
    .transferToHost(DataTransferMode.EVERY_EXECUTION, output);

try (TornadoExecutionPlan plan = new TornadoExecutionPlan(graph.snapshot())) {
    plan.execute();
}
```

`TaskGraph` declares **20 generic overloads** of `libraryTask`, one per arity, each taking a `LibraryTaskN<T1..TN>` functional interface from `TornadoFunctions` plus `N` typed arguments. Arity 20 is the ceiling; the widest factory actually shipped is `cublasSgemmStridedBatched` at 17 arguments.

The second argument is a *factory* — a static method reference that returns a `LibraryTaskDescriptor` rather than performing any work. The descriptor is a four-field record built with a fluent API:

Table 1 — `LibraryTaskDescriptor` fields (`tornado-api`)

Field	Set by	Meaning
<code>libraryName</code>	<code>withLibrary(String)</code>	Provider id, e.g. "nvidia/cublas". Matched against <code>TornadoLibraryProvider.libraryName()</code> at LAUNCH.
<code>functionName</code>	<code>withFunction(String)</code>	Key into the provider's dispatch registry, e.g. "cublasSgemv".
<code>parameters</code>	<code>withParameters(Object[])</code>	Positional arguments. Reference elements become device buffers; primitives are boxed constants.
<code>access</code>	<code>withAccess(Access[])</code>	Per-argument <code>READ_ONLY</code> / <code>WRITE_ONLY</code> / <code>READ_WRITE</code> . This is what the data-flow graph consumes.
<code>tuning</code>	<code>withTuning(Object)</code>	Opaque per-call options. <code>tornado-api</code> stays library-agnostic; <code>tornado-cublas</code> interprets it as <code>CuBlasOptions</code> .

The `Access[]` array is the load-bearing part. It is what lets a native call participate in TornadoVM's ordinary dependency analysis without the compiler ever seeing the callee. The cuBLAS factories compute it dynamically — note that `beta` changes the access class of the output operand:

```
private static Access[] readOnlyExcept(int numArgs, int outputIndex, float beta) {
    Access[] accesses = new Access[numArgs];
    Arrays.fill(accesses, Access.READ_ONLY);
    accesses[outputIndex] = (beta != 0.0f) ? Access.READ_WRITE : Access.WRITE_ONLY;
    return accesses;
}
```

With `beta != 0`, cuBLAS reads `C` before overwriting it, so the operand is marked `READ_WRITE` and TornadoVM keeps its device contents live. This is inferred from a *value* at graph-construction time, which is only possible because the factory runs on the host before the graph is snapshotted.

2. Anatomy of a library task

A library task is a `SchedulableTask` **without a sketch** — structurally the same trick as a pre-built task. It never enters the Graal-based JIT pipeline. What it does enter is everything else:

Table 2 — Lifecycle of a library task, stage by stage

Stage	Component	What happens
Construction	<code>TaskGraph.libraryTask</code>	Factory is invoked on the host; returns a <code>LibraryTaskDescriptor</code> . Shape validation happens here (e.g. CUTLASS rejects <code>k</code> not a multiple of 4 with a <code>TornadoRuntimeException</code>).
Graph build	<code>TornadoTaskGraph</code>	Wrapped in a <code>LibraryTask</code> ; its <code>Access[]</code> joins the data-flow graph exactly like a kernel's parameter accesses.
Bytecode gen	<code>TornadoVMGraphCompiler</code>	Emits ordinary <code>ALLOC</code> / <code>TRANSFER_HOST_TO_DEVICE</code> / <code>LAUNCH</code> / <code>TRANSFER_DEVICE_TO_HOST</code> . Arguments are pushed as <code>PUSH_CONSTANT_ARGUMENT</code> or <code>PUSH_REFERENCE_ARGUMENT</code> .
Pre-compile	<code>withPreCompilation()</code>	Provider <code>prepare()</code> runs: native plans, descriptors and workspaces are created <i>before</i> any capture region opens.
LAUNCH	<code>TornadoVMInterpreter.executeLibraryLaunch</code>	Reference args resolved to raw device pointers, constants unboxed from the pool, provider located, context fetched or created, <code>NVTX</code> range opened, <code>dispatch()</code> called.
Teardown	<code>LibraryRegistry.destroyContexts</code>	On execution-plan close, every context for that plan id is passed to its provider's <code>destroyContext()</code> .

The interpreter's argument-resolution loop is the seam between the two worlds:

```

for (int i = 0; i < numArgs; i++) {
    final byte argType = bytecodeResult.get();
    final int argIndex = bytecodeResult.getInt();
    if (argType == TornadoVMBytecodes.PUSH_CONSTANT_ARGUMENT.value()) {
        callArgs[i] = constants.get(argIndex);
    } else if (argType == TornadoVMBytecodes.PUSH_REFERENCE_ARGUMENT.value()) {
        final DataObjectState globalState = resolveGlobalObjectState(argIndex);
        final XPUDeviceBufferState objectState = globalState.getDeviceBufferState(interpreterDevice);
        callArgs[i] = objects.get(argIndex);
        // Pointer to the first data element on the device, past the array header
        devicePointers[i] = objectState.getXPUBuffer().toBuffer() + TornadoNativeArray.ARRAY_HEADER;
        isReference[i] = true;
    }
}
}

```

Everything the provider receives is packaged into a `LibraryInvocation`: `getArg(i)` for boxed scalars, `getDevicePointer(i)` for buffers, `isReference(i)`, `getContext()`, `getTuning()`, and `isCapturing()` — the last of which lets a provider refuse an operation that would be illegal inside a CUDA Graph capture region.

3. The provider SPI

Providers are discovered with `java.util.ServiceLoader` from `LibraryRegistry`, lazily and once per JVM (double-checked locking on a `volatile List`). Adding a library therefore requires **no change to `tornado-runtime`**.

Table 3 — `TornadoLibraryProvider`, the six methods a provider implements

Method	Contract
<code>libraryName()</code>	Returns the provider id string matched against the descriptor.
<code>canHandle(device)</code>	Capability gate. Every shipped provider tests <code>device instanceof TornadoNativeStreamSupport</code> — i.e. CUDA only.
<code>createContext(device, planId)</code>	Creates the native handle, binds it to the plan's CUstream, allocates a workspace. Called once per (<i>library, device, execution plan</i>).
<code>prepare(descriptor, context)</code>	Idempotent pre-capture hook. Builds and caches per-shape plans/descriptors/workspaces so that <code>dispatch()</code> allocates nothing.
<code>dispatch(fn, invocation)</code>	The actual call. Positional marshalling from the invocation to the native entry point.
<code>destroyContext(context)</code>	Frees plans, workspaces, and the handle when the execution plan closes.

Two supporting types complete the SPI. `LibraryContext` is a bare marker interface — each provider subclasses it with whatever state it needs (handle, stream, plan cache, workspace pointer and size). `TornadoNativeStreamSupport` is implemented by the CUDA backend device and exposes `getNativeStream(planId)`, `getNativeContext(planId)`, and the NVTX range calls.

4. Device pointers, layout, and the alignment consequence

TornadoVM's off-heap arrays (`FloatArray`, `HalfFloatArray`, `BFloat16Array`, `DoubleArray`, `IntArray`, `ByteArray`) store a **24-byte header** ahead of the payload. The device pointer handed to a native library is `buffer + TornadoNativeArray.ARRAY_HEADER`.

Structural consequence. Because 24 is not a multiple of 16, the payload pointer can never be 16-byte aligned even when the buffer base is. TornadoVM therefore guarantees only **8-byte alignment**. This propagates directly into the CUTLASS kernels, which are instantiated with `kAlign = 4` (four halves = 8 bytes) instead of the 8 (16-byte, 128-bit) that CUTLASS FP16 tensor-core kernels normally use — and which is why the FP16 factories reject shapes where `k` or `n` is not a multiple of 4. It is also why cuBLASLt FP8, which requires leading dimensions that are multiples of 16 bytes, is the one path with an explicit alignment caveat in its javadoc.

Storage order differs per provider and is not normalized by TornadoVM:

Table 4 — Memory layout per provider

Provider	Order	Consequence for row-major TornadoVM arrays
cuBLAS / cuBLASLt	Column-major	Caller compensates. GEMV: pass <code>CUBLAS_OP_T</code> . GEMM: swap operands — row-major $C = A \cdot B$ is column-major $C_{cm} = B_{cm} \cdot A_{cm}$. No data is moved; the cost is purely cognitive.
CUTLASS	Row-major	Direct match. All operands declared <code>cutlass::layout::RowMajor</code> in the <code>.cu</code> .
cuSPARSE	Row-major dense	Direct match — dense operands created with <code>CUSPARSE_ORDER_ROW</code> .
cuDNN	NCHW / BHSD	Direct match; packed strides computed in the shim.
cuFFT	Interleaved complex	(<code>re</code> , <code>im</code>) pairs packed into a <code>FloatArray/DoubleArray</code> .

5. Stream binding and ordering

Each provider binds its native handle to the backend's CUstream during `createContext` — `cublasSetStream_v2`, `cublasLtMatmul`'s stream argument, `cufftSetStream`, `cudnnSetStream`, `cusparseSetStream`; CUTLASS takes the stream per call. Everything therefore runs **in order on one stream**: JIT kernels, H2D/D2H transfers, and native calls, with no host synchronization at the boundary.

The interpreter adds an explicit guard for the non-capture case. When dependency tracking is active it enqueues a marker on the producer events before the native call, ensuring the library sees completed inputs. During capture this is skipped deliberately — a single in-order stream already orders the work, and waiting on pre-capture events would invalidate the capture.

6. `prepare()` and CUDA Graph capture safety

Device allocation is illegal inside a CUDA Graph capture region. Every library that creates per-shape plans or workspaces — cuDNN, CUTLASS, cuFFT, cuSPARSE, cuBLAS, cuBLASLt — does that work in `prepare()`, which the interpreter invokes in the pre-compilation pass before capture begins. `prepare()` is idempotent via a plan-cache lookup, so the per-call `dispatch()` allocates nothing and is capture-safe.

```
try (TornadoExecutionPlan plan = new TornadoExecutionPlan(graph.snapshot())) {
    plan.withCUDAgraph(); // iteration 0 captures; the rest replay
    for (int i = 0; i < iterations; i++) {
        plan.execute();
    }
}
```

One provider goes further and uses the capture flag defensively. cuSPARSE pre-allocates an 8 MiB workspace in `prepare()`, but the true external-buffer size depends on the sparse structure and is only known at dispatch, from `cusparseSpMV_bufferSize`. If a matrix needs more than the pre-allocated amount *and* `invocation.isCapturing()` is true, the provider raises a clear error rather than corrupting the capture:

```
long needed = CusparseNativeLib.spmvBufferSize(context.handle, rows, cols, nnz, ...);
context.growWorkspace(needed, invocation.isCapturing()); // throws if growth is needed mid-capture
```

7. Context lifetime and caching

`LibraryRegistry` keys contexts on a three-part record:

```
private record ContextKey(String libraryName, TornadoXPUDevice device, long executionPlanId) {}
private static final Map<ContextKey, LibraryContext> CONTEXTS = new ConcurrentHashMap<>();
```

Per-plan scoping is what makes cuBLASLt's heuristic cache and cuDNN's convolution plan cache correct: a plan owns its shapes. Two execution plans using the same device get independent handles, workspaces and caches, and closing one frees only its own.

Table 5 — Per-context cached state, by provider

Provider	Handle / plan cache	Workspace policy
cuBLAS	<code>cublasHandle_t</code>	User-requested via <code>CuBlasOptions.withWorkspace(bytes)</code> ; grow-only, 256-byte aligned, pre-allocated in <code>prepare()</code> , freed with the context.
cuBLASLt	<code>cublasLtHandle_t</code> + cache of (shape, types, epilogue) → <code>matmul plan(desc + layouts + heuristic algo)</code>	Fixed 32 MiB , allocated in <code>createContext</code> .
cuFFT	Plan cache keyed by (kind, dim1, dim2); plans created with <code>cufftPlanId/cufftPlan2d</code>	cuFFT-internal work area (allocated at plan creation, hence in <code>prepare()</code>).
cuDNN	<code>cudaDnnHandle_t</code> ; per-shape <code>ConvPlan</code> (tensor/filter/conv/output descriptors + algo + workspace size); per-shape SDPA plan from <code>cudaDnn-frontend</code>	Grow-only device workspace shared by conv and SDPA.
cuSPARSE	<code>cusparseHandle_t</code>	8 MiB default, grow-only, refuses to grow during capture.
CUTLASS	Stream captured at <code>createContext</code>	Per-shape workspace sized in <code>prepare()</code> from the kernel's own <code>get_workspace_size</code> .

8. Profiling and observability

Library tasks report `TASK_KERNEL_TIME` alongside `BACKEND`, `DEVICE` and `METHOD`, next to regular tasks, under `--enableProfiler console`. The timing is **host-side**, bracketed by stream markers that are waited on, because the CUDA-C backend does not yet expose device-event timestamps for this path:

```
final boolean profileCall = TornadoOptions.isProfilerEnabled() && !insideCaptureRegion;
if (profileCall) {
    int preEvent = interpreterDevice.enqueueMarker(planId);
    interpreterDevice.resolveEvent(planId, preEvent).waitForEvents(planId);
    profilerStartTime = System.nanoTime();
}
```

Two consequences follow. Profiling a library task *synchronizes the stream twice per call*, so it is a measurement mode, not a production one. And it is disabled entirely while capturing, since synchronizing would invalidate the capture.

Independently of the profiler, every dispatch is wrapped in an NVTX range named `<library>/<function>` (e.g. `nvidia/cutlass/cutlassHgemm`), so native calls appear as labelled spans on the Nsight Systems timeline next to the backend's own kernel and transfer ranges. This is a no-op without a profiler attached.

```
nsys profile --trace=cuda,nvtx -o run tornado -m <module>/<MainClass>
nsys stats --report cuda_gpu_trace run.nsys-rep # library kernels + JIT kernels, same Strm
```

9. Execution-plan controls that matter for hybrid plans

Table 6 — `TornadoExecutionPlan` options with library-task-specific relevance

Option	Backends	Relevance
<code>withCUDAgraph()</code>	CUDA	Captures the whole region — transfers, JIT kernels, native calls — and replays with one <code>cuGraphLaunch</code> .
<code>withPreCompilation()</code>	all	Runs every provider's <code>prepare()</code> up front. Prerequisite for graph capture.
<code>transferToDevice(Object...)</code>	all	Uploads large stable inputs (weights, tables) without running the plan, so the first execution does not pay the upload.
<code>withMemoryLimit("8GB")</code>	all	Worth setting explicitly — library workspaces (32 MiB for cuBLASLt, 8 MiB for cuSPARSE, unbounded grow-only for cuDNN) sit alongside operands.
<code>withIntraPlanConcurrency()</code>	CUDA	Routes DAG-independent work to separate role streams. Off by default.
<code>withStagedTransfers()</code>	CUDA	Stages non-batched read-only transfers through pinned host memory.

Option	Backends	Relevance
<code>withWarmUpIterations(n)</code>	all	Runs the plan n times before the measured run.
<code>withBatch(...)</code>	—	Rejected. <code>executeLibraryLaunch</code> throws if <code>batchThreads != 0</code> . Use a library's own batched entry point instead.

10. Failure modes

Table 7 — How each failure surfaces

Condition	Where	Behaviour
Vendor library absent at class-init	<code>*NativeLib</code> static init	<code>SymbolLookup</code> (or <code>System.loadLibrary</code>) comes back empty; provider reports <code>UNSUPPORTED</code> at runtime. The build still succeeds — every binding is self-guarding.
Non-CUDA default device	<code>LibraryRegistry.findProvider</code>	Provider found by name but <code>canHandle</code> false → <code>TornadoRuntimeException</code> naming the device and telling the user to select a compatible backend.
Module not on module path	<code>LibraryRegistry.findProvider</code>	<code>TornadoRuntimeException</code> : "No provider found for library ... ensure the corresponding TornadoVM library module is on the module path."
Unknown function name	provider dispatch registry	Explicit error from the provider's registry lookup.
Bad shape (CUTLASS FP16)	factory, at graph build	Fails fast on the host, before any device work: "requires k and n to be multiples of 4 (8-byte operand alignment); got $n=...$, $k=...$ ".
Native error status	provider dispatch	<code>TornadoRuntimeException</code> carrying the decoded status string (<code>CUBLAS_STATUS_INVALID_VALUE</code> , <code>CUSPARSE_STATUS_ARCH_MISMATCH</code> , ...).
Workspace growth during capture	cuSPARSE context	Explicit refusal with remediation advice (run once without CUDA graphs, or reduce the problem size).
<code>withBatch</code> on a library task	interpreter	<code>TornadoRuntimeException</code> naming the offending task id.

On the "silent UNSUPPORTED" phrasing. `HYBRID_API_GUIDE.md` says library tasks are "silently reported as `UNSUPPORTED` on OpenCL/Metal". In the code the outcome is a thrown `TornadoRuntimeException` with a descriptive message; what is silent is the *build*, not the run. The JUnit suites turn this into a skip through a JUnit assumption on the default device's backend.

Part II — What is supported

Six providers ship in this build. Every entry point below was read from the factory classes and cross-checked against the provider dispatch registry and the native binding, so the tables state what the code does rather than what the documentation claims.

Table 8 — Provider inventory

Provider id	Java module	Native binding	Entry points	JUnit tests	Domain
nvidia/cublas	tornado-cublas	FFM → libcublas	7	14	Dense BLAS: GEMV, GEMM, batched GEMM, mixed-precision GemmEx
nvidia/cublasLt	tornado-cublas	FFM → libcublasLt	5	6	Plan-based matmul with fused epilogues; FP8
nvidia/cufft	tornado-cufft	FFM → libcufft	8	8	1D/2D FFTs, FP32 and FP64
nvidia/cudnn	tornado-cudnn	FFM → libcudnn + JNI shim (cudnn-jni)	7	11	DL primitives + fused flash attention
nvidia/cuspars	tornado-cuspars	FFM → libcuspars	2	11	CSR sparse × dense products
nvidia/cutlass	tornado-cutlass	JNI, nvcc-compiled (cutlass-jni)	10	25	Row-major GEMM with fused activation epilogues
Total			39	75	

Four of the six bind through `java.lang.foreign` alone — a `SymbolLookup` resolved at class-init and one downcall handle per entry point, with no native module and nothing gated behind the `cuda-backend` Maven profile beyond the Java module itself. Only cuDNN (the cudnn-frontend C++ SDPA shim) and CUTLASS (device code that must go through `nvcc`) still carry a compiled artifact.

11. cuBLAS — nvidia/cublas

Factories on `uk.ac.manchester.tornado.cublas.CuBlas`. Column-major, so callers pass a transpose op (GEMV) or swap operands (GEMM).

Table 9 — cuBLAS entry points, complete

Factory	Native symbol	Types	Args	Semantics and configuration
<code>cublasSgemv(op, m, n, alpha, A, lda, x, incx, beta, y, incy)</code>	<code>cublasSgemv_v2</code>	<code>FloatArray</code>	11	$y = \alpha \cdot \text{op}(A) \cdot x + \beta \cdot y$. Output index 9, <code>READ_WRITE</code> when $\beta \neq 0$.
<code>cublasSgemm(transa, transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)</code>	<code>cublasSgemm_v2</code>	<code>FloatArray</code>	13	$C = \alpha \cdot \text{op}(A) \cdot \text{op}(B) + \beta \cdot C$. Output index 11.
<code>cublasSgemmTF32(...)</code>	<code>cublasSgemm_v2 + cublasSetMathMode</code>	<code>FloatArray</code>	13	Delegates to <code>cublasSgemm</code> with <code>withTuning(CUBLAS_TF32_TENSOR_OP_MATH)</code> . Math mode is set before the call and restored to <code>CUBLAS_DEFAULT_MATH</code> after. ~10-bit mantissa multiply, FP32 accumulate.
<code>cublasSgemmStridedBatched(transa, transb, m, n, k, alpha, A, lda, strideA, B, ldb, strideB, beta, C, ldc, strideC, batchCount)</code>	<code>cublasSgemmStridedBatched</code>	<code>FloatArray</code> , long strides	17	$C[i] = \alpha \cdot \text{op}(A[i]) \cdot \text{op}(B[i]) + \beta \cdot C[i]$. Operand i at <code>base + i · stride</code> in one flat array. Output index 13.
<code>cublasGemmExFP16(...)</code>	<code>cublasGemmEx</code>	<code>HalfFloatArray</code> in/out	13	FP16 in and out, <code>CUBLAS_COMPUTE_32F</code> accumulate on tensor cores.
<code>cublasGemmExFP16FP32(...)</code>	<code>cublasGemmEx</code>	<code>HalfFloatArray</code> A,B → <code>FloatArray</code> C	13	FP16 inputs, FP32 output — keeps accumulation precision at the boundary. Standard inference config.
<code>cublasGemmExBF16(...)</code>	<code>cublasGemmEx</code>	<code>BFloat16Array</code> in/out	13	BF16 in and out, FP32 accumulate. BF16 keeps FP32's exponent range (no ±65504 overflow boundary).

PER-CALL TUNING

`CuBlasOptions` is attached through the opaque `withTuning` field and carries exactly two knobs:

Knob	Values	Applied as
<code>withMathMode(...)</code>	<code>CUBLAS_DEFAULT_MATH</code> (0), <code>CUBLAS_PEDANTIC_MATH</code> (1), <code>CUBLAS_TF32_TENSOR_OP_MATH</code> (3)	<code>cublasSetMathMode</code> before the call, restored to default after.
<code>withWorkspace(bytes)</code>	any size	<code>cublasSetWorkspace_v2</code> . Per-(device, plan), grow-only, freed with the plan. Needed for run-to-run reproducibility across streams and for device-launched graphs.

Four enum classes ship in `tornado-cublas`: `CuBlasOperation` (N/T/C), `CuBlasMathMode` (3 values), `CublasComputeType` (11 values), and `CudaDataType` (16 values). The latter two are **far wider than what the factories use** — the compute-type enum declares INT32, FP64 and the FAST_16F/FAST_16BF families that no entry point currently reaches. They are scaffolding for the roadmap, not capability.

12. cuBLASLt — nvidia/cublasLt

A separate provider inside the same module. It builds a matmul *plan* — descriptor, three matrix layouts, and a heuristic-selected algorithm — per unique (shape, types, epilogue), caches it in the context, and executes with a 32 MiB workspace. All five entry points use `CUBLAS_COMPUTE_32F` with scale type `CUDA_R_32F`, because alpha and

beta are passed as host floats.

Table 10 — cuBLASLt entry points, complete

Factory	A / B type	C type	Epilogue	Notes
ltMatmulFP32	CUDA_R_32F	CUDA_R_32F	DEFAULT (1)	13 args. Plain cublasLtMatmul.
ltMatmulFP16	CUDA_R_16F	CUDA_R_16F	DEFAULT (1)	13 args. FP32 tensor-core accumulation.
ltMatmulFP8	CUDA_R_8F_E4M3 in a ByteArray	CUDA_R_16F	DEFAULT (1)	13 args. Requires the TN form (<code>transa = OP_T, transb = OP_N</code>) and leading dimensions that are multiples of 16 bytes. Hardware FP8 path: Ada sm_89 / Hopper.
ltMatmulBiasFP16	CUDA_R_16F	CUDA_R_16F	BIAS (4)	14 args. $C = \text{op}(A) \cdot \text{op}(B) + \text{bias}$, bias broadcast over the m rows of the column-major result.
ltMatmulGeluBiasFP16	CUDA_R_16F	CUDA_R_16F	GELU_BIAS (36)	14 args. $C = \text{GELU}(\text{op}(A) \cdot \text{op}(B) + \text{bias})$, tanh approximation. The fully fused transformer MLP block.

Twelve `cublasLt*` symbols are bound: `Create`, `Destroy`, `MatmulDescCreate/Destroy/SetAttribute`, `MatrixLayoutCreate/Destroy`, `MatmulPreferenceCreate/Destroy/SetAttribute`, `MatmulAlgoGetHeuristic`, and `Matmul` itself. The `CuBlasLtEpilogue` enum declares six values (`DEFAULT`, `RELU`, `BIAS`, `RELU_BIAS`, `GELU`, `GELU_BIAS`) but the dispatch registry reaches only three.

13. cuFFT — `nvidia/cufft`

Plans are created with `cufftPlan1d` / `cufftPlan2d` and cached per (`kind`, `dim1`, `dim2`). All eight factories share the same access pattern: { `READ_ONLY`, `WRITE_ONLY`, `READ_ONLY`, `READ_ONLY` } — in, out, and two dimension scalars. Complex data is interleaved (`re`, `im`). Inverse transforms follow cuFFT semantics and are **unnormalized**: $\text{inverse}(\text{forward}(x)) = n \cdot x$.

Table 11 — cuFFT entry points, complete

Factory	Native exec	Plan type	Array	Buffer sizing
<code>cufftForwardC2C(in, out, n, batch)</code>	<code>cufftExecC2C(dir -1)</code>	CUFFT_C2C 1D	FloatArray	In and out: $2 \cdot n \cdot \text{batch}$ floats.
<code>cufftInverseC2C(in, out, n, batch)</code>	<code>cufftExecC2C(dir +1)</code>	CUFFT_C2C 1D	FloatArray	As above; unnormalized.
<code>cufftForwardR2C(in, out, n, batch)</code>	<code>cufftExecR2C</code>	CUFFT_R2C 1D	FloatArray	In $n \cdot \text{batch}$ reals $\rightarrow$ out $2 \cdot (n/2+1) \cdot \text{batch}$ floats (non-redundant Hermitian half).
<code>cufftInverseC2R(in, out, n, batch)</code>	<code>cufftExecC2R</code>	CUFFT_C2R 1D	FloatArray	In $2 \cdot (n/2+1) \cdot \text{batch}$ floats (must be Hermitian) $\rightarrow$ out $n \cdot \text{batch}$ reals.
<code>cufftForwardZ2Z(in, out, n, batch)</code>	<code>cufftExecZ2Z(dir -1)</code>	CUFFT_Z2Z 1D	DoubleArray	FP64. $2 \cdot n \cdot \text{batch}$ doubles.
<code>cufftInverseZ2Z(in, out, n, batch)</code>	<code>cufftExecZ2Z(dir +1)</code>	CUFFT_Z2Z 1D	DoubleArray	FP64; unnormalized.
<code>cufftForward2dC2C(in, out, nx, ny)</code>	<code>cufftExecC2C(dir -1)</code>	CUFFT_C2C 2D	FloatArray	$n_x \times n_y$ row-major grid, $2 \cdot n_x \cdot n_y$ floats.
<code>cufftInverse2dC2C(in, out, nx, ny)</code>	<code>cufftExecC2C(dir +1)</code>	CUFFT_C2C 2D	FloatArray	Unnormalized: $\text{inverse}(\text{forward}(x)) = n_x \cdot n_y \cdot x$.

14. cuDNN — `nvidia/cudnn`

Two distinct generations sit behind one Java surface. Four factories use the **legacy** compute functions (FP32, NCHW); one uses the **graph API** through a `cudnn-frontend` v1.25.0 C++ shim compiled into `libtornado-cudnn`.

Table 12 — cuDNN entry points, complete

Factory	cuDNN call	Precision	Fixed configuration
<code>cudnnSoftmax(in, out, rows, cols)</code>	<code>cudnnSoftmaxForward</code>	FP32, NCHW	CUDNN_SOFTMAX_ACCURATE, INSTANCE mode. Rows normalized independently — the attention-score shape.
<code>cudnnRelu(in, out, size)</code>	<code>cudnnActivationForward</code>	FP32	Element-wise; tensor described as $1 \times 1 \times 1 \times \text{size}$. Three of cuDNN's activation modes are reachable; no coefficient parameter is exposed.
<code>cudnnSigmoid(in, out, size)</code>			
<code>cudnnTanh(in, out, size)</code>			
<code>cudnnMaxPool2d(in, out, n, c, h, w, window, stride)</code>	<code>cudnnPoolingForward</code>	FP32, NCHW	MAX only, square window, square stride, no padding . Output spatial dims $(h - \text{window})/\text{stride} + 1$.
<code>cudnnConv2d(in, filter, out, n, c, h, w, k, r, s, pad, stride)</code>	<code>cudnnConvolutionForward</code>	FP32, NCHW / KCRS	CUDNN_CROSS_CORRELATION, fixed algorithm IMPLICIT_PRECOMP_GEMM, dilation hard-coded to (1,1) , group count 1, square pad and stride. Output $(n, k, (h+2 \cdot \text{pad}-r)/\text{stride}+1, (w+2 \cdot \text{pad}-s)/\text{stride}+1)$.
<code>sdpaForward(q, k, v, o, b, h, sQ, sKv, d, scale, causal)</code>	graph API <code>sdpa()</code> via <code>cudnn-frontend</code>	FP16 I/O, FP32 intermediate and compute	Packed BHSD: $Q/O = [b, h, sQ, d]$, $K/V = [b, h, sKv, d]$, strides $\{h \cdot s \cdot d, s \cdot d, d, 1\}$. <code>set_generate_stats(false)</code> $\rightarrow$ inference only. Optional causal mask. Built with heuristic mode A. Constraints: d multiple of 8, $d \leq 256$, Ampere or newer.

The `h` parameter is shared between Q and K/V. The shim builds `dim_q = {b, h, s_q, d}` and `dim_kv = {b, h, s_kv, d}` from the same `h`, so grouped-query and multi-query attention (kv-heads $\neq$ q-heads) cannot be expressed — consistent with the roadmap, which marks C2 as in progress.

15. cuSPARSE — `nvidia/cusparse`

The narrowest provider: two operations, both over the modern generic API (`cusparseSpMV` / `cusparseSpMM` with `cusparseCreateCsr` + `CreateDnVec` / `CreateDnMat` descriptors).

Table 13 — cuSPARSE entry points, complete

Factory	Native call	Semantics and fixed configuration
<code>cusparseSpMV(rows, cols, nnz, csrRowOffsets, csrColInd, csrValues, x, y)</code>	<code>cusparseSpMV</code>	$y = A \cdot x$. A is $\text{rows} \times \text{cols}$ CSR, x length cols , y length rows (index 7, <code>WRITE_ONLY</code>).
<code>cusparseSpMM(rows, k, n, nnz, csrRowOffsets, csrColInd, csrValues, b, c)</code>	<code>cusparseSpMM</code>	$C = A \cdot B$. A is $\text{rows} \times k$ CSR, B dense $k \times n$ row-major, C dense $\text{rows} \times n$ (index 8, <code>WRITE_ONLY</code>).

Everything else is fixed in the provider, not parameterised:

- **Format:** CSR only — `CUSPARSE_INDEX_32I` for both offsets and column indices, `CUSPARSE_INDEX_BASE_ZERO`.
- **Type:** `CUDA_R_32F` throughout. `csrRowOffsets` / `csrColInd` are `IntArray`; values and dense operands are `FloatArray`.
- **Transpose:** `CUSPARSE_OPERATION_NON_TRANSPOSE` for both operands, hard-coded.
- **Scalars:** `alpha = 1.0f`, `beta = 0.0f`, passed as literals from the provider. The native layer accepts them as parameters, but no factory exposes them.
- **Algorithm:** `CUSPARSE_SPMV_ALG_DEFAULT` / `CUSPARSE_SPMM_ALG_DEFAULT`.
- **Ordering:** dense operands use `CUSPARSE_ORDER_ROW`, matching TornadoVM arrays.

16. CUTLASS — `nvidia/cutlass`

Row-major throughout, so there is no transposition to reason about. Kernels are compiled ahead of time by `nvcc` from `tornado-cutlass.cu`, which fetches header-only CUTLASS v3.5.1 through CMake `FetchContent`.

Table 14 — CUTLASS entry points, complete

Factory	Element type	Op class	Computes
<code>cutlassSgemm(m, n, k, α, A, B, β, C)</code>	<code>FloatArray</code> FP32	<code>OpClassSimt</code>	$C = \alpha \cdot A \cdot B + \beta \cdot C$. Alignment 1 — no shape constraint . The correctness baseline.
<code>cutlassHgemm(m, n, k, α, A, B, β, D)</code>	<code>HalfFloatArray</code> FP16	<code>OpClassTensorOp</code>	$D = \alpha \cdot A \cdot B + \beta \cdot D$, FP32 accumulate. Requires k , n multiples of 4.
<code>cutlassBgemm(m, n, k, α, A, B, β, C)</code>	<code>BFloat16Array</code> BF16	<code>OpClassTensorOp</code>	Same shapes and constraint as FP16 (bfloat16 is also 2 bytes). FP32 accumulate; FP32 exponent range.
<code>cutlassHgemmBatched(m, n, k, α, A, B, β, C, batchCount)</code>	<code>HalfFloatArray</code> FP16	<code>OpClassTensorOp</code>	<code>GemmUniversalMode::kBatched</code> . Batch strides fixed at $m \cdot k$, $k \cdot n$, $m \cdot n$ — matrices packed contiguously. Rejects <code>batchCount</code> ≤ 0 .
<code>cutlassGemmBiasRelu(m, n, k, A, B, bias, D)</code>	<code>HalfFloatArray</code> FP16	<code>OpClassTensorOp</code>	$D = \text{relu}(A \cdot B + \text{bias})$, <code>LinearCombinationRelu</code>
<code>cutlassGemmBiasGelu(...)</code>			$D = \text{gelu}(A \cdot B + \text{bias})$, <code>LinearCombinationGELU</code>
<code>cutlassGemmBiasSilu(...)</code>			$D = \text{silu}(A \cdot B + \text{bias})$, <code>LinearCombinationSilu</code> — the SwiGLU gate activation of the LLaMA/Qwen families
<code>cutlassGemmBiasSigmoid(...)</code>			$D = \text{sigmoid}(A \cdot B + \text{bias})$, <code>LinearCombinationSigmoid</code>
<code>cutlassGemmBiasTanh(...)</code>			$D = \text{tanh}(A \cdot B + \text{bias})$, <code>LinearCombinationGeneric<Tanh></code>
<code>cutlassGemmBiasHardSwish(...)</code>			$D = \text{hardswish}(A \cdot B + \text{bias})$, <code>LinearCombinationHardSwish</code>

THE SINGLE KERNEL CONFIGURATION

All FP16 and BF16 kernels are one instantiation of `cutlass::gemm::device::GemmUniversal`, fixed at compile time:

Template parameter	Value	Consequence
Architecture tag	<code>cutlass::arch::Sm80</code>	Built as <code>sm_80</code> SASS plus <code>compute_80</code> PTX. Runs natively on all Ampere/Ada 8.x; JIT-compile from PTX on Hopper. Override with <code>CUDA_ARCH=<cc></code> .
ThreadblockShape	<code>GemmShape<128, 128, 32></code>	One tile size for every shape — no per-problem tile selection.
WarpShape	<code>GemmShape<64, 64, 32></code>	Standard Ampere HMMMA configuration.
InstructionShape	<code>GemmShape<16, 8, 16></code>	
kStages	3	Three-stage software pipeline.
kAlign (A and B)	4	Four halves = 8 bytes. Forced by TornadoVM's 24-byte header. This is the k , n multiple-of-4 constraint.
Layouts	<code>RowMajor</code> $\times 3$	No column-major or mixed-layout instantiation exists.
Swizzle	<code>GemmIdentityThreadblockSwizzle</code>	Default identity swizzle.

The fused-epilogue kernels use a specific trick: the length- n bias vector is supplied as the `C` source operand with `ldc = 0`, so every output row reads the same vector, and the epilogue computes $\text{act}(\alpha \cdot A \cdot B + \beta C)$ with $\alpha = \beta = 1$ **hard-coded**:

```
template <typename Gemm>
static int gemmBiasAct(int m, int n, int k, jlong dA, jlong dB, jlong dBias, jlong dD, ...) {
    typename Gemm::Arguments args(
        cutlass::gemm::GemmUniversalMode::kGemm, {m, n, k}, 1, {1.0f, 1.0f},
        (void const *) dA, (void const *) dB, (void const *) dBias, (void *) dD,
        0, 0, 0, 0, k, n, 0, n); // ldc = 0 -> row broadcast
    return runGemm<Gemm>(args, (void *) workspace, (cudaStream_t) stream);
}
```


This is why the six bias-activation factories take no `alpha` / `beta` arguments, unlike `cutlassSgemm` / `cutlassHgemm` / `cutlassBgemm` .

17. Composition patterns

The value of the mechanism is not any single call but the ability to interleave. Four patterns are exercised by the test suites:

Table 15 — Composition patterns and their gating tests

Pattern	Shape	Gating test
JIT → library → JIT	Native call sandwiched between kernels on shared buffers	<code>TestCuBlas#testMixedPrePostTasks</code> , <code>TestCutlass#testSgemmWithJitPreAndPost</code> , <code>TestCuspars#testSpMVWithJitPreAndPost</code>
Library → library chain	Two native calls, intermediate stays on device	<code>TestCutlass#testChainedSgemm</code> , <code>#testTwoLayerFusedMlp</code> , <code>TestCuDnn#testConvReluPoolPipeline</code>
Shared buffers across graphs	<code>persistOnDevice</code> / <code>consumeFromDevice</code> — a producer graph's native result consumed by a second graph, no host round trip	<code>TestCuBlas#testSharedBufferAcrossTaskGraphs</code> , <code>TestCutlass#testSharedBufferAcrossTaskGraphs</code>
Mixed precision in one graph	FP32 and FP16 library tasks side by side	<code>TestCutlass#testMixedPrecisionGraph</code>
Under CUDA Graph capture	Any of the above, captured once and replayed	<code>TestCuBlas#testMixedTasksWithCudaGraph</code> , <code>TestCuDnn#testConv2dWithCudaGraph</code> , <code>TestCuspars#testSpMVWithCudaGraph</code> , <code>TestCutlass#testGemmWithCudaGraph</code> , <code>#testFusedMlpWithCudaGraph</code>

A representative deep pipeline — a two-layer feed-forward block as two fused CUTLASS tasks, with nothing returning to the host between them:

```
new TaskGraph("ffn")
    .transferToDevice(DataTransferMode.EVERY_EXECUTION, x, w1, b1, w2, b2)
    .libraryTask("l1", Cutlass::cutlassGemmBiasRelu, m, n1, k1, x, w1, b1, h)
    .libraryTask("l2", Cutlass::cutlassGemmBiasGelu, m, n2, n1, h, w2, b2, y)
    .transferToHost(DataTransferMode.EVERY_EXECUTION, y);
```

Part III — Comparison with the actual NVIDIA libraries

18. Method, and the two questions that get confused

"How does it compare with the actual NVIDIA libraries" splits into two questions with opposite answers, and keeping them apart is the whole point of this section.

Question	Short answer	Why
Is it as fast? For an operation that is bound, does the TornadoVM path reach the library's own throughput?	Essentially yes	TornadoVM does not reimplement anything. It calls the real <code>libcublas</code> , <code>libcufft</code> , <code>libcudnn</code> , <code>libcuspars</code> , <code>libcublasLt</code> , and real CUTLASS device code, on the real stream, with real device pointers. The only additions are a hash lookup, a downcall, an NVTX push/pop, and a pointer arithmetic. Section 29 quantifies the exceptions.
Is it as capable? How much of each library's API can you reach?	A small fraction	39 entry points across six libraries. Whole levels of BLAS, all of cuDNN's backward pass, five of six cuSPARSE formats, and CUTLASS's entire configuration space are absent. Sections 19–26 enumerate this.

The denominators below are counted in **API families** — a family being one mathematical operation regardless of its type-suffixed variants. Counting cuBLAS's `gemm` once rather than four times (S/D/C/Z), or eight (adding the `_64` forms), is the reading most favourable to TornadoVM, and even so the coverage is thin. Family counts are enumerated from the public NVIDIA API references and are approximate at the margins (whether `gemm3m` is its own family, and so on); they are stated to establish an order of magnitude, not a precise ratio.

19. cuBLAS

Table 16 — cuBLAS coverage by API section

Section	NVIDIA families	Bound	Coverage	What is bound / what is not
Level 1 (vector)	~13	0	0%	Nothing. <code>axpy</code> , <code>dot</code> , <code>nrm2</code> , <code>scal</code> , <code>copy</code> , <code>swap</code> , <code>asum</code> , <code>amax/amin</code> , <code>rot/rotg/rotrm/rotrmg</code> — all absent.
Level 2 (matrix-vector)	~23	1	~4%	gemv (FP32 only). Absent: <code>gbmv</code> , <code>symv</code> , <code>sbmv</code> , <code>spmv</code> , <code>trmv</code> , <code>tbbmv</code> , <code>tpmv</code> , <code>trsv</code> , <code>tbsv</code> , <code>tpsv</code> , <code>ger</code> , <code>syr</code> , <code>syr2</code> , <code>spr</code> , <code>spr2</code> , and all seven Hermitian routines.
Level 3 (matrix-matrix)	~15	2	~13%	gemm , gemmStridedBatched . Absent: <code>symm</code> , <code>syrk</code> , <code>syr2k</code> , <code>syrkx</code> , <code>trmm</code> , <code>trsm</code> , <code>trsmBatched</code> , <code>gemmBatched</code> (pointer-array form), <code>gemm3m</code> , and all Hermitian forms.
BLAS-like extensions	~19	1	~5%	gemmEx (three typed factories). Absent: <code>geam</code> (would give an on-device transpose — directly useful at the row/column-major seam), <code>dgmm</code> , <code>gemmBatchedEx</code> , <code>gemmStridedBatchedEx</code> , <code>axpyEx/dotEx/scalEx/nrm2Ex/rotEx</code> , and the batched LAPACK-ish family (<code>getrfBatched</code> , <code>getriBatched</code> , <code>getrsBatched</code> , <code>matinvBatched</code> , <code>geqrfBatched</code> , <code>gelsBatched</code>).
Total	~70	4	~6%	

DATA-TYPE COVERAGE

cuBLAS ships S/D/C/Z variants of nearly every routine, plus the `Ex` family's per-operand `cudaDataType`. TornadoVM reaches four element types and no complex ones:

Type	Bound	Notes
FP32 (S)	yes	<code>gemv</code> , <code>gemm</code> , <code>gemmStridedBatched</code> , TF32 math mode
FP16	yes	<code>gemmEx</code> , FP16 → FP16 and FP16 → FP32
BF16	yes	<code>gemmEx</code> , BF16 → BF16
FP64 (D)	no	No <code>DoubleArray</code> path in cuBLAS at all. <code>dgemm</code> is a trivial clone of <code>sgemm</code> and is roadmap item B1.
Complex (C, Z)	no	No complex array type participates. Rules out the entire Hermitian half of Levels 2 and 3.
INT8 / INT32	no	<code>CublasComputeType.CUBLAS_COMPUTE_32I</code> is declared in the enum but unreachable. Roadmap C3.

HANDLE-LEVEL CONFIGURATION

cuBLAS control	Bound	Consequence
<code>cublasSetStream</code>	yes	The whole mechanism depends on it.
<code>cublasSetMathMode</code>	yes	DEFAULT / PEDANTIC / TF32, per call, restored after.
<code>cublasSetWorkspace</code>	yes	Grow-only, per (<i>device</i> , <i>plan</i>).
<code>cublasSetPointerMode</code>	no	The most consequential gap. With the default <code>CUBLAS_POINTER_MODE_HOST</code> , <code>alpha/beta</code> are host floats. Under CUDA Graph capture they are <i>baked into the captured graph by value</i> , so a scaling factor cannot vary between replays without re-capturing. Device pointer mode would also let a preceding JIT task compute <code>alpha</code> — and is the only way to bind the scalar-output Level-1 routines (<code>dot</code> , <code>nrm2</code> , <code>asum</code>) without synchronizing the stream and breaking capture. Roadmap A3/E4.
<code>cublasSetAtomicMode</code>	no	Only affects <code>symv/hemv</code> , neither of which is bound.
<code>cublasGemmAlgo_t</code> selection	no	Every <code>gemmEx</code> call uses the default algorithm. Note that <code>CUBLAS_GEMM_ALGO*_TENSOR_OP</code> autotuning would not be capture-safe anyway.

cuBLAS control	Bound	Consequence
64-bit API (cublas*_64)	no	All dimensions and leading dimensions are <code>int</code> . TornadoVM off-heap segments can exceed <code>INT_MAX</code> elements, so this is a real ceiling, not a theoretical one. Roadmap G1.
cublasXt (multi-GPU)	no	Marked not planned in the roadmap: its host-pointer API conflicts with TornadoVM's device-buffer model.

20. cuBLASLt

cuBLASLt's surface is not a routine list but a descriptor system: one `cublasLtMatmul` plus roughly 30 exported functions for building descriptors, layouts, preferences, and algorithm configurations. TornadoVM binds 12 of them, enough to build and execute a plan, and none of the algorithm-introspection or matrix-transform surface.

Table 17 — cuBLASLt: what a plan can and cannot express through TornadoVM

Axis	Reachable	Not reachable
Epilogue	DEFAULT, BIAS, GELU_BIAS (3)	The rest of the enum, roughly 16 values: RELU, RELU_BIAS, GELU, GELU_AUX, GELU_AUX_BIAS, and the entire backward family (DRELU, DGELU, DRELU_BGRAD, DGELU_BGRAD, BGRADA, BGRADB). RELU and RELU_BIAS are already declared in <code>CuBlaSLtEpilogue</code> but have no factory — roadmap F3b.
Compute type	CUBLAS_COMPUTE_32F only	16F, 64F, 32I, and the FAST_16F / FAST_16BF / FAST_TF32 variants. The restriction is structural: <code>alpha/beta</code> are host floats, so the scale type must be <code>CUDA_R_32F</code> .
Element types	FP32, FP16, FP8 E4M3	BF16, FP8 E5M2, INT8. <code>CUDA_R_8F_E5M2</code> is declared in the enum and accepted by the FP8 type check, but no factory produces it.
FP8 scaling	Nothing	A real fidelity gap. Production FP8 matmul sets <code>A_SCALE_POINTER</code> , <code>B_SCALE_POINTER</code> , <code>D_SCALE_POINTER</code> and <code>AMAX_D_POINTER</code> on the matmul descriptor — that per-tensor scaling is what makes 8-bit numerically usable. <code>LtMatmulFP8</code> sets none of them, so all scales are implicitly 1.0. The path exercises the FP8 tensor cores correctly but is not the FP8 workflow a practitioner would recognise.
Batching	Nothing	<code>BATCH_COUNT</code> / <code>STRIDED_BATCH_OFFSET</code> layout attributes are never set. Batched Lt matmul is unavailable — the only batched paths are <code>cublasSgemmStridedBatched</code> (FP32) and <code>cutlassHgemmBatched</code> (FP16).
Layout / ordering	Plain column-major	<code>CUBLASLT_ORDER_COL32</code> , <code>COL4_4R2_8C</code> , <code>COL32_2R_4R4</code> — the tensor-op-friendly orderings, and <code>cublasLtMatrixTransform</code> which converts between them.
Algorithm control	<code>MatmulAlgoGetHeuristic</code> , top result	<code>AlgoGetIds</code> , <code>AlgoInit</code> , <code>AlgoCheck</code> , <code>AlgoConfigSetAttribute</code> — i.e. no explicit tile/split-k/stage search. The heuristics cache API (<code>HeuristicsCacheGetCapacity</code>) is also unbound; TornadoVM caches plans itself, per execution plan.
Workspace	Fixed 32 MiB	Not user-tunable. It matches NVIDIA's own recommendation for Ada/Hopper, but a plan with many concurrent contexts pays it repeatedly.

21. cuFFT

Table 18 — cuFFT coverage by axis

Axis	Bound	Not bound
Transform types	C2C, R2C, C2R, Z2Z (4 of 6)	D2Z, Z2D — double-precision real transforms. <code>cufftExecD2Z/Z2D</code> are simply not among the nine bound symbols.
Dimensionality	1D (all four types), 2D (C2C only)	3D entirely (<code>cufftPlan3d</code> unbound). 2D for R2C/C2R/Z2Z.
Plan construction	<code>cufftPlan1d</code> , <code>cufftPlan2d</code>	<code>cufftPlanMany</code> and <code>cufftMakePlanMany</code> — no advanced strided/embedded layouts, so batches must be contiguous and unit-stride. This rules out FFTs over a non-innermost axis of a multidimensional array without an explicit transpose kernel first.
Precision	FP32, FP64	FP16 and BF16 via <code>cufftXtMakePlanMany</code>.
Callbacks	—	<code>cufftXtSetCallback</code> / <code>cufftXtSetJITCallback</code>. These fuse pre/post-processing into the FFT kernel itself; TornadoVM's answer is a separate JIT task, which is composable but not fused.
Work area	Implicit	<code>cufftSetAutoAllocation</code> / <code>cufftSetWorkArea</code> — the FFT work area is allocated by cuFFT at plan creation, outside TornadoVM's memory accounting.
Multi-GPU	—	The whole <code>cufftXt</code> descriptor API.

Where the composition story wins anyway. The absent callback API is the case where TornadoVM's model substitutes cleanly: `FrequencyFilterExample` runs cuFFT R2C → a JIT kernel that zeroes high bins → cuFFT C2R → a JIT normalize, all on shared device buffers on one stream. A cuFFT callback would fuse the filter into the transform and save a round trip to global memory; the TornadoVM version does not, but it is four lines of ordinary Java rather than a device-side function pointer.

22. cuDNN

cuDNN is where the coverage ratio is worst and the reason is architectural. The library has two APIs: the legacy per-operation functions, and the v9 graph API. TornadoVM binds four legacy forward ops and exactly one graph-API pattern.

Table 19 — cuDNN coverage

Area	Bound	Not bound
Activations	ReLU, sigmoid, tanh (forward)	ELU, CLIPPED_RELU, SWISH, IDENTITY; the coefficient parameter; all backward passes.
Softmax	ACCURATE + INSTANCE, forward	FAST, LOG modes; CHANNEL mode; backward.

Area	Bound	Not bound
Pooling	MAX, square window, no padding, forward	Average pooling (both padding conventions), asymmetric windows and strides, padding, Nd pooling, backward.
Convolution	2D forward, FP32, NCHW, fixed IMPLICIT_PRECOMP_GEMM	The other seven forward algorithms and cudnnFindConvolutionForwardAlgorithm autotuning; NHWC (the tensor-core-preferred layout); FP16/BF16/INT8; dilation $\neq$ 1; grouped and depthwise convolution; 3D convolution; backward data and backward filter.
Attention	Fused SDPA forward, FP16, causal or unmasked, inference	GQA/MQA (kv-heads $\neq$ q-heads); padding, sliding-window and bias masks; dropout; ragged/variable-length offsets; paged KV-cache attention; FP8 SDPA; SDPA backward.
Normalization	—	Nothing. No batch norm, no LayerNorm, no RMSNorm — the last being the natural companion to SDPA in a LLaMA block, and roadmap D1.
Tensor ops	—	cudnnAddTensor (broadcast bias), cudnnOpTensor, cudnnReduceTensor. Bias-add in the CNN pipeline is done by a JIT task instead.
Other	—	Dropout, LRN, the RNN v8 API (explicitly out of scope for a transformer-focused effort), fused ConvBiasAct, NormAddRelu.
Training	—	No backward operation of any kind is bound. This is an inference-only binding.

The deprecation problem. Every legacy compute function TornadoVM binds — `cudnnSoftmaxForward`, `cudnnActivationForward`, `cudnnPoolingForward`, `cudnnConvolutionForward` — has been **deprecated since cuDNN 9.0**. They remain fully functional (validated on 9.23) but are slated for removal in favour of the graph API. The repository's own roadmap tracks this as item F6: re-implement the v1 ops as graph-API patterns behind the same Java factories now that the cudnn-frontend integration exists. The Java API surface would be unaffected; the risk is entirely on the binding's future compilability against a newer cuDNN.

23. cuSPARSE

Table 20 — cuSPARSE coverage by axis

Axis	Bound	Coverage	Not bound
Generic-API operations	SpMV, SpMM	2 of ~9	SpGEMM (sparse $\times$ sparse, and its reuse variant), SpSV and SpSM (triangular solve — the core of iterative solvers and preconditioners), SDDMM (sampled dense-dense, central to sparse attention), SpMMOp, Sparse-to-Dense and Dense-to-Sparse conversion, and the sparse-vector routines (axpby, gather, scatter, rot).
Sparse formats	CSR	1 of ~6	CSC, COO, BSR, Blocked-ELL (the format for structured 2:4 sparsity), Sliced-ELLPACK. Conversions between formats are also unbound, so a caller holding COO must convert on the host.
Index types	32-bit, zero-based	1 of 4	64-bit indices (CUSPARSE_INDEX_64I); one-based indexing.
Value types	FP32	1 of ~6	FP16, BF16, FP64, complex FP32/FP64, INT8.
Transpose modes	NON_TRANSPOSE	1 of 3	TRANSPOSE, CONJUGATE_TRANSPOSE — hard-coded in the provider for both operands.
Scalars	$\alpha = 1, \beta = 0$	fixed	Arbitrary α/β . The native layer already takes them as parameters; only the factory signature withholds them. This is the cheapest gap in the whole report to close.
Algorithms	ALG_DEFAULT	1 of ~4	SPMV_CSR_ALG1/ALG2, SPMM_CSR_ALG1/ALG2/ALG3 — the deterministic and load-balanced variants.

24. CUTLASS

CUTLASS is not an API with a function list; it is a template metaprogramming library whose configuration space is effectively unbounded. The honest comparison is therefore not "how many functions" but "how much of the design space is instantiated".

Table 21 — CUTLASS: the instantiated slice vs. the template space

Dimension	TornadoVM instantiates	CUTLASS offers
Architecture	Sm80 only	Sm70, Sm75, Sm80, Sm89, Sm90 (Hopper warp-specialized, TMA, persistent kernels), Sm100. On Hopper, TornadoVM's <code>compute_80</code> PTX JITs and runs, but as an Ampere kernel — it cannot use TMA or the Hopper-class GEMM schedules that account for much of Hopper's advantage.
Kernel API generation	CUTLASS 2.x device: <code>:GemmUniversal</code>	CUTLASS 3.x <code>CollectiveBuilder</code> / CuTe, which is where the Hopper and Blackwell kernels live.
Tile configuration	One: 128 $\times$ 128 $\times$ 32 threadblock, 64 $\times$ 64 $\times$ 32 warp, 16 $\times$ 8 $\times$ 16 instruction, 3 stages	The full tile/stage/swizzle search space, plus the CUTLASS profiler that picks per-problem. A single tile is right for large square problems and wrong for skinny ones — exactly the shapes LLM decoding produces.
Alignment	kAlign = 4 (64-bit loads)	kAlign = 8 (128-bit loads), the normal setting for FP16 tensor-core GEMM. Blocked by TornadoVM's 24-byte array header.
Layouts	RowMajor $\times$ RowMajor $\times$ RowMajor	Any combination of RowMajor/ColumnMajor; plus interleaved and tensor layouts.
Element types	FP32 (SIMT), FP16, BF16	TF32, FP8 (E4M3/E5M2), FP6, FP4, INT8, INT4, binary, and mixed-input GEMM (e.g. FP16 $\times$ INT4, the weight-only-quantization primitive).
Operations	GEMM, batched GEMM	Implicit-GEMM convolution (fprop/dgrad/wgrad), grouped GEMM (the MoE primitive), sparse (2:4) GEMM, Syrk/Trmm, ranked contractions, fused multi-head attention examples.
Epilogues	6 activations + plain linear combination	The full <code>LinearCombination*</code> catalogue plus user-written epilogue visitor trees (EVT), per-channel scaling, residual add, and split-k reductions.
Scheduling	Identity swizzle, no split-k	Split-k, stream-k, and the threadblock swizzles that matter for occupancy on non-square problems.

25. Semantic fidelity gaps

Distinct from "not bound at all": these are operations that *are* bound but whose TornadoVM form is narrower than the native call, in ways a caller could reasonably not expect.

Table 22 — Where a bound operation is narrower than its native counterpart

Operation	Native capability	TornadoVM form
<code>cusparseSpMV / SpMM</code>	Arbitrary α and β ; $\text{op}(A) \in \{N, T, CT\}$; six formats; six value types; four algorithm choices	$\alpha = 1, \beta = 0$, N only, CSR only, FP32 only, default algorithm. $\beta = 0$ means the accumulate form $y = \alpha Ax + \beta y$ is unavailable — a caller wanting accumulation must add a JIT task.
<code>cudaDnnConvolutionForward</code>	Eight algorithms plus heuristic selection; α/β blending; dilation; groups; NCHW and NHWC; FP32/FP16/INT8	One algorithm, no blending, dilation fixed at (1,1), one group, NCHW, FP32. The fixed algorithm is the largest single performance concession in the binding (see §29).
<code>cudaDnnPoolingForward</code>	MAX and two AVG modes; asymmetric window, stride and padding; Nd	MAX, square window, square stride, zero padding, 2D.
<code>cuDNN SDPA</code>	GQA/MQA; four mask kinds; dropout; ragged offsets; paged KV cache; FP8; training	Shared head count for Q and KV; causal or nothing; FP16; inference (<code>set_generate_stats(false)</code>).
<code>cuBLASLtMatmul FP8</code>	Per-tensor A/B/D scale pointers and AMAX output — the mechanism that makes FP8 numerically viable	No scale attributes set; implicit scale 1.0. Correct as an FP8 tensor-core exercise, incomplete as an FP8 workflow.
CUTLASS fused epilogues	Arbitrary α and β in the <code>LinearCombination</code> parameters	$\alpha = \beta = 1$ hard-coded in <code>gemmBiasAct</code> ; bias delivered as the C operand with <code>ldc = 0</code> . Six of the ten factories therefore take no scaling arguments at all.
<code>cutlassHgemmBatched</code>	Independent batch strides per operand	Strides fixed at $m \cdot k, k \cdot n, m \cdot n$ — matrices must be packed contiguously, so a broadcast operand (one shared B across the batch, common in attention) cannot be expressed.
<code>cuBLAS GEMM/GEMV under capture</code>	Device pointer mode lets α/β vary per replay	Host pointer mode only; scalars are baked into the captured graph.

26. Cross-cutting structural limits

Table 23 — Limits that apply to every provider

Limit	Origin	Effect
8-byte pointer alignment	24-byte <code>TornadoNativeArray</code> header	CUTLASS FP16/BF16 restricted to 64-bit vector loads and to k, n multiples of 4. <code>cuBLASLt</code> FP8's 16-byte leading-dimension requirement becomes the caller's problem. This is the deepest structural constraint in the design, and it is not fixable inside a provider.
32-bit dimensions	All factory signatures take <code>int</code>	No access to the <code>cuBLAS*_64</code> entry points or 64-bit <code>cuSPARSE</code> indices, even though TornadoVM segments can exceed <code>INT_MAX</code> elements.
Single device	<code>ContextKey</code> is per-device, but task graphs are single-device	No multi-GPU library task. <code>cuBLASXt</code> is explicitly ruled out; NCCL has no binding at all.
<code>withBatch</code> incompatibility	Interpreter rejects <code>batchThreads != 0</code>	TornadoVM's own out-of-core batching cannot be combined with library tasks. Batching must come from the library (<code>cuBLASSgemmStridedBatched</code> , <code>cutlassHgemmBatched</code>) or from an explicit loop.
Host-timed profiling	CUDA-C backend exposes no device timestamps for this path	<code>TASK_KERNEL_TIME</code> costs two stream synchronizations per call and is disabled under capture. NVTX ranges are the accurate alternative.
CUDA-only	Every <code>canHandle</code> requires <code>TornadoNativeStreamSupport</code>	No library task runs on the OpenCL, Metal, PTX or SPIR-V paths. There is no <code>rocmBLAS</code> / <code>oneMKL</code> analogue, and no fallback to a JIT implementation of the same operation — a graph written with library tasks is not portable.
No JIT fallback	By design	A missing vendor library is an exception, not a slower path. Portable code must branch on device capability at graph-construction time.

27. NVIDIA libraries with no binding at all

Table 24 — Absent from this build

Library	Status	Notes
<code>cuTENSOR</code>	Branch only	An <code>nvidia/cutensor</code> provider for tensor contractions and einsum exists on branch hybrid-cutensor and is not part of this build . The only trace in the tree is a comment in <code>CUDACommandQueue</code> listing it as an NVTX-range example.
NCCL	Not implemented	No module, no provider, no reference anywhere in <code>tornado-*</code> . Collective communication would also require multi-device task graphs, which TornadoVM does not yet have.
CUB / CuTe	Not applicable	Header-only <i>device</i> libraries. These would plug into the CUDA-C backend's NVRTC compilation, not the library-task path, since they are called from inside a kernel rather than from the host.
<code>cuSOLVER</code>	Not implemented	No binding. Dense and sparse factorizations, eigensolvers and SVD are entirely absent — as are <code>cuBLAS</code> 's own batched LU/QR/least-squares routines, which are the nearest available substitute.
<code>cuRAND</code>	Not implemented	No binding. Random number generation stays in JIT kernels.
<code>nvJPEG</code> , <code>NPP</code> , <code>cuDSS</code> , <code>cuQuantum</code>	Not implemented	Outside the project's stated scope.

Documentation drift, in TornadoVM's favour. The two ROADMAP files understate what is now shipped. `tornado-cublas/ROADMAP.md` marks C4 (BF16 `GemmEx`) and C5 (FP8) as *blocked on missing tornado-api types* and F4 (`cuBLASLt` FP8) as not started — but `cuBLASGemmExBF16`, `cutlassBgemm` and `ltMatmulFP8` all exist in the code, and `BFloat16Array` and `ByteArray`-backed FP8 are wired through. Similarly, `tornado-cudnn/README.md` lists the graph API under "not bound yet" while `sdpaForward` ships. Treat the roadmaps as design records, not as a coverage statement.

Part IV — Performance

28. Measured numbers

Every figure below is published in the repository, measured on an **NVIDIA GeForce RTX 4090** (Ada, SM 8.9) with CUDA 12.6 and cuDNN 9.23. They are collected here unchanged; nothing was re-measured for this report, and no GPU was available in the environment it was written in.

Table 25 — SGEMM ladder (`BenchmarkSgemm` , FP32/TF32/FP16, TFLOP/s)

Size	Naive @Parallel	KernelContext (tiled)	cuBLAS FP32	cuBLAS TF32	cuBLAS FP16	FP16 vs tiled
1024	4.0	3.9	26.1	28.2	44.1	11.3×
2048	5.1	6.0	48.3	62.8	120.6	20.4×
4096	5.3	5.8	57.0	81.1	160.6	27.4×

The repository states that 160.6 TFLOP/s is the RTX 4090's peak FP16 tensor throughput and 81 TFLOP/s its TF32 peak — i.e. at 4096 the library task is running at the hardware ceiling, with TornadoVM's dispatch overhead invisible. Each step across that table is a factory-name swap in the task graph.

Table 26 — SGEMV, memory-bound (`MatrixVectorRowMajorWithCuBLas` , 8192×2048, 120 iterations)

Variant	Time	Bandwidth	vs sequential Java
Sequential Java	8.551 ms	7.9 GB/s	1.0×
@Parallel kernel (naive)	3.309 ms	20.3 GB/s	2.6×
KernelContext (workgroup-per-row)	0.164 ms	410 GB/s	52×
cuBLAS SGEMV library task	0.116 ms	578 GB/s	74×

This is the instructive counterweight to Table 25: on a memory-bound kernel cuBLAS is only **1.4×** the best hand-written TornadoVM kernel, against 6–27× on the compute-bound one. The payoff from a library task tracks how much tensor-core and tiling expertise the operation rewards.

Table 27 — Fused epilogue (`BenchmarkLtFusedMlp` , FP16, TFLOP/s)

Size	Unfused: GemmEx + JIT bias/GELU	Fused: cuBLASLt GELU_BIAS	Fusion speedup
1024	17.5	29.4	1.68×
2048	73.8	98.3	1.33×
4096	132.1	147.3	1.11×

Fusion pays most at small and medium sizes — the launch-and-memory-bound regime that LLM decoding lives in — and converges toward parity as the GEMM itself starts to dominate.

Table 28 — cuDNN convolution (`BenchmarkConv2d` , FP32, 3×3, pad 1)

Layer (NCHW)	TornadoVM JIT direct conv	cuDNN library task	Speedup
8×64×56×56, k=64	3.8 TFLOP/s	11.6 TFLOP/s	3.1×
16×128×28×28, k=128	4.0 TFLOP/s	21.1 TFLOP/s	5.3×

Table 29 — Fused flash attention (`BenchmarkSdpa` , causal)

Shape (b, h, s, d)	JIT attention kernel (FP32)	cuDNN fused SDPA (FP16)	Speedup
4, 16, 1024, 128	90.7 ms — 0.19 TFLOP/s	0.268 ms — 64.2 TFLOP/s	339×
1, 32, 2048, 128	203.3 ms — 0.17 TFLOP/s	0.407 ms — 84.3 TFLOP/s	499×

These are the largest ratios in the repository, and they are honest but need reading carefully: they compare a *fused FP16 flash-attention kernel* against an *unfused FP32 JIT attention kernel* that materializes the full score matrix. Roughly half a step of the gap is precision and tensor cores, the rest is the algorithmic difference between flash attention and the naive formulation. Results are cross-validated against the FP32 reference on the same FP16-rounded inputs.

Table 30 — Remaining published figures

Benchmark	Configuration	Result
BenchmarkFft	n = 65536, 20 iterations	Sequential Java DFT 228,819 ms · TornadoVM JIT DFT kernel 63.4 ms · cuFFT 0.080 ms — 793× the JIT kernel.
BenchmarkCutlassGemm	1024×1024, FP16	CUTLASS tensor-core ≈ 39 TFLOP/s , 7.7× the tiled KernelContext kernel. Fused GEMM+bias+GELU ≈ 1.28× the unfused CUTLASS GEMM followed by a JIT epilogue task.
TestCuBlasSgemmTF32	4096	82.6 TFLOP/s, 14.2× the tiled KernelContext kernel, maximum relative error ≈ 1e-4.

An internal inconsistency worth knowing about. `docs/source/hybrid-api.rst` quotes cuBLAS SGEMM as "24 / 46 / 51 TFLOP/s at 1024 / 2048 / 4096"; `tornado-cublas/README.md` quotes 26.1 / 48.3 / 57.0 for the same benchmark and sizes, and `tornado-cublas/ROADMAP.md` quotes 51 TFLOP/s at 4096 alongside a TF32 figure of 82.6 where the README says 81.1. These are separate measurement runs recorded at different times rather than a contradiction, but any figure quoted downstream should name its source file.

29. Where parity holds, and where it breaks

Because TornadoVM invokes the vendor library rather than reimplementing it, the default expectation is parity. The per-call overhead added on the dispatch path is: one `ConcurrentHashMap` lookup for the context, one linear scan over the (six-element) provider list, N pointer additions, one hash lookup in the provider's dispatch registry, an NVTX push/pop, and an FFM downcall or JNI transition. All of this is host-side and measured in nanoseconds against kernels measured in microseconds to milliseconds. Under CUDA Graph capture even that disappears from the replay path.

Four situations break the parity expectation, in descending order of importance.

1. CUDNN CONVOLUTION RUNS A FIXED ALGORITHM

The binding hard-codes `CUDNN_CONVOLUTION_FWD_ALGO_IMPLICIT_PRECOMP_GEMM` and never calls `cudaFindConvolutionForwardAlgorithm` or the v7 heuristic query. cuDNN's central value proposition is that it picks among eight algorithms per shape; a caller who fixes one is using cuDNN as a kernel library rather than as cuDNN. Combined with FP32/NCHW — rather than FP16/NHWC, the tensor-core-preferred combination — the measured 11.6 and 21.1 TFLOP/s sit well below what the same GPU reaches on cuBLAS FP32 GEMM (48–57 TFLOP/s, Table 25), let alone its FP16 tensor path. The repository's own README names algorithm autotuning and the FP16 tensor-core path as "the next big steps", so this is a known position rather than an oversight.

2. CUTLASS RUNS ONE TILE CONFIGURATION AT REDUCED ALIGNMENT

A derived comparison, using two different benchmarks on the same GPU at the same size and precision: `BenchmarkCutlassGemm` reports ≈ 39 TFLOP/s for FP16 at 1024, while `BenchmarkSgemv` reports 44.1 TFLOP/s for cuBLAS FP16 at the same size. The CUTLASS path is therefore roughly **0.9× cuBLAS** on this shape. Two causes are visible in the source and both are structural rather than incidental:

- `kAlign = 4` — 64-bit operand loads instead of the 128-bit loads a normal FP16 tensor-core CUTLASS kernel uses, forced by the 24-byte array header (§4).
- A single $128 \times 128 \times 32$ tile for every problem, where cuBLAS dispatches among many tuned kernels and the CUTLASS profiler would select per shape.

These are not a controlled A/B — the two benchmarks differ in harness and validation — so treat 0.9× as an indication, not a measurement. The direction is nonetheless what the kernel configuration predicts. CUTLASS's advantage here is not raw GEMM throughput but the fused epilogues and the row-major layout; on a plain GEMM, cuBLAS is the better call.

3. PROFILING CHANGES WHAT IT MEASURES

Enabling `--enableProfiler` inserts two stream markers per library task and waits on both. For a short call this can dominate the measurement. NVTX ranges under `nsys` are the non-perturbing alternative and are always on.

4. HOST-POINTER SCALARS UNDER CUDA GRAPH REPLAY

Not a throughput issue but a correctness trap: α and β are captured by value, so a replayed graph silently reuses the values from capture time. There is no diagnostic; the caller must re-capture or avoid varying scalars.

What TornadoVM adds that the C API does not have. The comparison so far has been one-directional. Against a hand-written C program calling the same libraries, the TornadoVM path adds: automatic buffer lifetime and residency across the JIT $\leftrightarrow$ native boundary; dependency-driven ordering derived from `Access[]` rather than hand-placed events; CUDA Graph capture of the whole mixed region for free; a uniform profiler and NVTX labelling across JIT and native work; and the ability to write the surrounding pre/post-processing in Java instead of CUDA C. For the operations that *are* bound, that is a strictly better place to call them from.

Part V — Engineering surface

30. Build, install, and packaging

One command builds everything: `make BACKEND=cuda`, which activates the `cuda-backend` Maven profile. The five Java modules always compile. What differs is whether a native artifact is also produced.

Table 31 — Build and install requirements per provider

Provider	Native module	Runtime dependency	How obtained · library search order
cuBLAS	none	CUDA toolkit	Nothing to install. FFM lookup: <code>libcublas.so.12</code> → <code>.so.11</code> → <code>.so</code> → <code>cublas64_12.dll</code> → <code>libcublas.dylib</code> ; also loads <code>libcudart</code> .
cuBLASLt	none	CUDA toolkit	<code>libcublasLt.so.12</code> → <code>.so.11</code> → <code>.so</code> → <code>cublasLt64_12.dll</code> → <code>.dylib</code> .
cuFFT	none	CUDA toolkit	<code>libcufft.so.11</code> → <code>.so.10</code> → <code>.so</code> → <code>cufft64_11.dll</code> → <code>.dylib</code> .
cuSPARSE	none	CUDA toolkit	<code>libcusparses.so.12</code> → <code>.so.11</code> → <code>.so</code> → <code>cusparse64_12.dll</code> → <code>.dylib</code> .
cuDNN	<code>cudnn-jni</code>	libcudnn 9 (separate install)	<code>apt install libcudnn9-cuda-12 libcudnn9-dev-cuda-12</code> . FFM lookup <code>libcudnn.so.9</code> → <code>.so.8</code> → ...; the JNI shim also fetches <code>cudnn-frontend v1.25.0</code> . Note: the Debian package installs headers under the multiarch path <code>/usr/include/x86_64-linux-gnu</code> , which the CMake hints cover.
CUTLASS	<code>cutlass-jni</code>	CUDA 12.0+, C++ toolchain	Header-only CUTLASS v3.5.1 fetched by CMake <code>FetchContent</code> ; nothing to install. Compiled for <code>sm_80 SASS + compute_80 PTX</code> ; override with <code>CUDA_ARCH=<cc></code> (e.g. <code>CUDA_ARCH=89</code>). On CUDA < 12 the native library is skipped and the provider reports <code>UNSUPPORTED</code> .

The design principle throughout is **self-guarding**: a missing library never fails the build. The `SymbolLookup` (or the native `.so` load) comes back empty and that provider reports `UNSUPPORTED` at runtime. The launcher adds the provider modules to `--add-modules` automatically whenever the CUDA backend is present: `tornado.cublas`, `tornado.cufft`, `tornado.cudnn`, `tornado.cusparses`, `tornado.cutlass`.

31. Test coverage

75 JUnit tests across five suites in `tornado-uitests`, all registered with `tornado-test` and all self-skipping through a JUnit assumption when the default device is not on the CUDA backend or the vendor library is unavailable. Every operation is validated against a sequential Java reference.

Table 32 — Test suites

Suite	Tests	What is covered beyond the plain operation
<code>cublas.TestCuBblas</code>	14	$\beta \neq 0$ read-write output, TF32 math mode, user workspace, <code>GemmEx</code> in FP16/BF16/FP16 → FP32, strided-batched, mixed pre/post JIT tasks, cross-graph shared buffers, CUDA Graph capture, ad-hoc and up-front weight upload.
<code>cublas.TestCuBblasLt</code>	6	FP32 and FP16 matmul, plan-cache reuse across shapes, BIAS and GELU_BIAS epilogues.
<code>cufft.TestCuFfft</code>	8	C2C round trip, R2C/C2R Hermitian round trip, Z2Z FP64, 2D transforms, capture/replay.
<code>cudnn.TestCuDnn</code>	11	Softmax, ReLU, tanh, max-pool, conv2d; the conv → JIT bias → ReLU → pool mixed pipeline; SDPA plain and causal; conv under CUDA Graph; up-front upload paths.
<code>cusparses.TestCusparses</code>	11	General, identity, diagonal, rectangular, empty-row and near-dense CSR matrices; SpMM; JIT pre/post; capture; multiple shapes sharing one context; repeated-execution stability.
<code>cutlass.TestCutlass</code>	25	The widest suite: all six fused epilogues individually, a SwiGLU feed-forward composite, batched HGEMM, BF16, rectangular shapes, chained GEMMs, a two-layer fused MLP, a deep mixed pipeline, cross-graph shared buffers, mixed precision in one graph, multiple shapes on one context, repeated-execution stability, capture of both a plain GEMM and a fused MLP, and a negative test (<code>testHgemmBadAlignmentShape</code>) asserting the alignment rejection.

Alongside these, each module ships `main()`-runnable examples and benchmarks that double as documentation — `TestCuBblasSgemv`, `FrequencyFilterExample`, `BenchmarkSgemm`, `BenchmarkSdpa`, `BenchmarkCutlassGemm` and the rest — each validating against sequential Java and printing throughput.

32. Writing a provider

The extension story is the strongest part of the design: adding a library touches **only the new module**. Four steps, mirroring `tornado-cudnn`:

1. **Factory** — a static method returning a `LibraryTaskDescriptor` with library name, function name, `Object[]` parameters, and the `Access[]` array.
2. **Provider** — implement `TornadoLibraryProvider`'s six methods; bind the stream in `createContext`, do all allocation in `prepare()`.
3. **Register** — provides ... with `MyProvider`; in `module-info.java` and the same class name in `META-INF/services/uk.ac.manchester.tornado.runtime.library.spi.TornadoLibraryProvider`.
4. **Bind** — a `SymbolLookup` plus downcall handles in `java.lang.foreign.A` `tornado-drivers/<lib>-jni` CMake module is needed *only* when the library requires actual compiled C/C++ (a header-only template library, or an ABI FFM cannot express) — and that path additionally requires wiring in the root `pom.xml`, the `tornado-drivers` profile, `tornado-assembly` (`assembly.xml` and `pom.xml`), and `tornado.py`'s `--add-modules` list.

Four of the six shipped providers took path (4) without a JNI module, which is the evidence that the FFM route is the default and the native module the exception.

Part VI — Assessment

33. Findings

WHAT THE DESIGN GETS RIGHT

1. **The `Access[]` insight.** Reusing TornadoVM's existing data-flow machinery by having a factory declare per-argument access — including deriving `READ_WRITE` from the runtime value of `beta` — is what lets an opaque native call participate in dependency analysis without the compiler knowing anything about it. Everything else follows from this.
2. **The `prepare()` / `dispatch()` split.** Hoisting every allocation into a pre-capture hook is what makes native library calls CUDA-Graph-capturable at all, and it is enforced uniformly across six providers with different allocation behaviours.
3. **Zero core-runtime coupling.** Six providers exist and `tornado-runtime` knows about none of them. `LibraryRegistry` is 60 lines of `ServiceLoader` and a `ConcurrentHashMap`.
4. **Self-guarding builds.** A missing vendor library degrades to a runtime `UNSUPPORTED`, never a build failure. This is what makes it viable to ship six optional NVIDIA dependencies in one SDK.
5. **Honest, comparative benchmarks.** Every benchmark measures against the best hand-written TornadoVM kernel, not only against sequential Java, and cross-validates results. The SGEMV table — where cuBLAS wins by only 1.4× — is included even though it is the least flattering number in the set.

THE LIMITS THAT MATTER MOST

1. **The 24-byte header caps alignment at 8 bytes.** This is the one constraint no provider can work around, and it costs measurable CUTLASS FP16 throughput while imposing a user-visible shape restriction. Any future FP8 or 128-bit-load path meets the same wall.
2. **Host pointer mode blocks a whole class of operations.** Without `CUBLAS_POINTER_MODE_DEVICE`, α and β cannot be produced on-device or varied across CUDA Graph replays, and the scalar-output Level-1 routines cannot be bound at all without breaking capture. Roadmap A3/E4 is the highest-leverage unblocking item in the tree.
3. **cuDNN's fixed convolution algorithm.** Using one of eight algorithms without heuristic selection forgoes cuDNN's main advantage, and the FP32/NCHW choice forgoes tensor cores on top of that.
4. **cuSPARSE's $\alpha=1$, $\beta=0$.** The narrowest provider is narrower than it needs to be — the native layer already takes both scalars as parameters; only the factory signature withholds them.
5. **Inference only, single device.** No backward operation anywhere, no multi-GPU, no NCCL. The binding targets inference and numerical kernels, not training.
6. **Not portable.** A graph containing a library task runs on CUDA or throws. There is no JIT fallback for the same operation, so portable code must branch at graph-construction time.

OVERALL

The right way to read this subsystem is as a **mechanism with a demonstration set**, not as a Java binding for the NVIDIA math libraries. The mechanism — shared device buffers, one stream, dependency-driven ordering, capture-safe preparation, `ServiceLoader` extensibility — is complete and uniformly applied, and it is the hard part. The 39 entry points on top of it are a curated slice chosen for LLM and CNN inference: GEMM in five precisions, fused MLP epilogues, flash attention, FFTs, CSR SpMV. Where a slice exists it runs at the vendor library's own speed, up to the hardware ceiling. Everywhere else, the honest answer to "does TornadoVM support *X*" is that adding *X* is a factory, a registry entry, a downcall handle and a test, in one module, with no core changes — which is precisely what the design was built to make true.

Compiled from `beehive-lab/TornadoVM` at commit `e06a2e5`. Primary sources: `HYBRID_API_GUIDE.md`; `docs/source/hybrid-api.rst`; the six `tornado-<lib>` modules and their READMEs and ROADMAPs; `tornado-drivers/{cudnn-jni,cutlass-jni}`; `tornado-runtime/.../runtime/library/`; `TornadoVMInterpreter.executeLibraryLaunch`; and the five suites under `tornado-unittests/.../unittests/`. NVIDIA-side API family counts are enumerated from the public cuBLAS, cuBLASLt, cuFFT, cuDNN, cuSPARSE and CUTLASS references and are approximate at the margins; they establish orders of magnitude, not exact ratios. No GPU was available while this report was written — all performance figures are quoted from the repository, not re-measured.